

DATA FEED INTEGRATION & Configurable Multi-Symbol Architecture Specification

*Kinetick · eSignal · IQFeed (DTN) — Technical deep-dive, tick field mapping,
adapter design, symbol count from 1 to 1000+, configurable architecture*

v1.0 | May 2026 | Engineering Reference

1. Data Provider Overview — Kinetick, eSignal, IQFeed

The Order Flow Engine must support all three data providers you use. Each has different connection protocols, tick field structures, historical data capabilities, symbol coverage, and limitations. This section documents all three in technical detail based on official documentation and research.

Provider	Owner	Protocol	Connection Type	Primary Use With NT	Tick Quality
Kinetick	NinjaTrader Group LLC (now Kraken)	Proprietary — native NinjaTrader protocol	Built directly into NT8; TCP/IP to Kinetick servers	NinjaTrader 8 primary feed — recommended by Valtos	Unfiltered, real-time; exchange direct for futures
eSignal	Intercontinental Exchange (ICE)	EFS (eSignal Formula Script) API; Desktop API; TCP/IP streaming	COM/ActiveX Desktop API; TCP socket; DDE link; C++ API	NinjaTrader 8 supported via eSignal connection adapter	Real-time streaming tick-by-tick; Level 1; Level 2; trade bid/ask separate
IQFeed (DTN)	DTN (Data Transmission Network)	TCP/IP socket — plain text CSV-like protocol; versioned protocol 6.2	Multiple TCP ports: Level1 (5009), Level2 (9200), History (9100), Admin (9300)	NinjaTrader 8 supported; also used standalone via API	True tick-by-tick — no filtering, aggregation. Exchange-direct Tier One vendor

2. Kinetick — Technical Specification

2.1 What Kinetick Is

Kinetick is NinjaTrader's own market data service, now owned by Kraken (acquired NinjaTrader May 2025 for \$1.5B). It is the preferred and tightest-integrated data feed for the NinjaTrader platform. Michael Valtos explicitly recommends Kinetick in his 'Trading Order Flow' book: 'I get my data from Kinetick. The quality is excellent and the price is affordable.'

- Exchange-direct data feed — writes directly to NT market data servers without an extra API layer
- Native NinjaTrader integration — no separate adapter needed; built into NT8 Connection Manager
- Unfiltered, real-time quotes for stocks, futures, and forex
- Free End-of-Day (EOD) service built into NT platform; paid real-time plans for live trading
- Provides failover redundancy — can be set as backup to brokerage account data feed
- Tick data: real-time bid/ask; historical tick data available (8 days during RTH, 180 days outside RTH)
- Historical bid/ask tick data: each tick assigned volume=1 in historical data (no historical volume per tick)
- Supported asset classes: Equities (NYSE, NASDAQ), Futures (CME, CBOT, NYMEX, COMEX, ICE), Forex

2.2 Kinetick Tick Data Fields

The following fields are delivered by Kinetick to NinjaTrader's OnMarketData() event handler. These are what the OFE Kinetick Adapter will receive and process:

Field Name	NT Data Type	Description	OFE Usage
Price	double	Last traded price at this tick	Primary: price level for bar accumulation
Volume	long	Number of contracts/shares at this tick (1 for historical bid/ask ticks)	Volume per price level — bid_vol or ask_vol
Time	DateTime	Timestamp of the tick (exchange time)	exchange_ts — converted to Unix nanoseconds
MarketDataType	enum	Last, Bid, Ask, DailyHigh, DailyLow, DailyVolume, LastClose, Opening, Opening, Settlement, Unknown	Determines if tick is trade (Last), bid quote (Bid), or ask quote (Ask)
Instrument	Instrument	The instrument this tick belongs to	Symbol routing — identifies which

			Symbol Worker receives this tick
IsReset	bool	True if the feed restarted and historical data follows	Engine state: flush current bar, restart from historical backfill
TickCount	int	Internal tick counter (not exchange sequence)	Secondary ordering — not used for Lee-Ready, only for debugging

2.2.1 MarketDataType Mapping for OFE

Kinetick MarketDataType	OFE Classification	Action in Symbol Worker
Last	TRADE — classify as BID or ASK via Lee-Ready	Accumulate bid_vol or ask_vol at this price. Update delta.
Bid	QUOTE — bid side	Update current best bid for Lee-Ready classification of next trade tick. NOT accumulated as volume.
Ask	QUOTE — ask side	Update current best ask for Lee-Ready classification. NOT accumulated as volume.
DailyVolume	SUMMARY — total session volume	Update session volume counter. Used for Volume Profile and VWAP denominator.
DailyHigh / DailyLow	SUMMARY — session extremes	Update session high/low tracking for context.
Opening / Settlement	SUMMARY	Store as session reference levels.
Unknown	DISCARD	Log and skip — do not process.

2.3 Kinetick Symbol Subscription & Limits

Plan	Monthly Cost	Symbol Limit	Real-Time Exchanges	Market Depth (DOM)
EOD (Free)	\$0	Unlimited	No — EOD only	No
Real-Time Base	~\$50/mo	100 symbols	Yes — base exchanges	Optional add-on (\$23/mo)
Real-Time 200	~\$70/mo	200 symbols	Yes	Optional add-on
Real-Time 500	~\$80/mo	500 symbols	Yes	Optional

				add-on
+ Exchange Fees	Varies	—	CME, NYSE, NASDAQ etc. add \$10–\$30/mo each	Depth add-on required per exchange

 *Kinetick symbol limits are SUBSCRIPTION limits — you pay per tier. The OFE engine's symbol count configuration must respect the user's Kinetick subscription limit. If the user configures 300 symbols but has the 200-symbol plan, the 301st subscription request will be silently rejected by Kinetick.*

2.4 Kinetick Adapter — OFE Integration Design

Since Kinetick is native to NinjaTrader, the OFE Kinetick Adapter runs INSIDE NinjaTrader 8 as an AddOn or Indicator that forwards tick data to the OFE engine via a local TCP socket or shared memory:

```
NinjaTrader 8
├─ Kinetick connection (built-in)
│   └─ OnMarketData() event → OFE NT Adapter (C# AddOn)
│       └─ Serialise tick → send to OFE Engine via localhost:7777 TCP
│           └─ OFE Tick Router → Symbol Worker for instrument
```

- The adapter subscribes to all instruments in the user's OFE watchlist using NT's AddDataSeries()
- Each OnMarketData() call is serialised to a compact binary format (FlatBuffers) and sent to OFE
- Connection: localhost loopback TCP — sub-microsecond overhead
- Backup: if OFE engine is not running, NT adapter logs ticks to a local file for replay on engine restart
- Symbol subscription changes (add/remove symbols to watchlist) trigger Subscribe/Unsubscribe calls to NT

3. eSignal — Technical Specification

3.1 What eSignal Is

eSignal (owned by Intercontinental Exchange / ICE since 2016, formerly Interactive Data Corporation) is a professional market data and charting platform. It provides real-time streaming data for global markets including stocks, futures, forex, and options across 95% of world exchanges.

- Owned by ICE (Intercontinental Exchange) — one of the world's largest exchange operators
- Current version: eSignal 21.4 (January 2024), Windows-based
- Powered by the ICE Consolidated Feed — a state-of-the-art, low-latency global data feed covering all asset classes
- Programming language: EFS (eSignal Formula Script) — JavaScript-based scripting for custom indicators
- APIs available: Desktop API (COM/ActiveX), C/C++ API, Java API, XML API, DDE link, QLink
- NinjaTrader 8 integration: supported via eSignal connection adapter (requires eSignal Premier or above)
- Tick data: real-time trade ticks + separate bid/ask quote ticks; Time & Sales window
- Historical tick data: up to 40 days via Extended Data subscription
- Global coverage: AMEX, NYSE, NASDAQ, CME, CBOT, COMEX, NYMEX, ICE, CBOE, EUREX, NYSE Liffe, LSE, Euronext, TSX, and 100+ more
- eSignal Classic: delayed/EOD only — does NOT provide tick-level data. Real-time product required for order flow.

3.2 eSignal Tick Data Fields

eSignal delivers tick data via Time & Sales (T&S) which separates Trades, Bids, and Asks into distinct message types. The OFE eSignal Adapter must handle all three:

eSignal T&S Field	Data Type	Description	OFE Mapping
Time	DateTime (ms precision)	Timestamp of the tick — exchange time	exchange_ts — converted to Unix nanoseconds. Note: ms precision, not ns.
Date	Date	Calendar date of the tick	Combined with Time for full datetime
Price / Last	double	Execution price for trades; bid price for bid quotes; ask price for ask quotes	price field in TickRecord
Size	long	Volume. Stocks: 1 unit = 100	volume field. Note

		shares. Futures: 1 unit = 1 contract.	stocks multiplier: $\text{raw_size} \times 100 = \text{actual shares}$
Type	enum {Trade, Bid, Ask}	Whether this T&S row is a completed trade, a bid quote update, or an ask quote update	Trade → Lee-Ready classify → bid_vol or ask_vol. Bid/Ask → update prevailing quote.
Bid	double	Current best bid price (in summary header above T&S)	Used to update prevailing bid for Lee-Ready classification
Ask	double	Current best ask price (in summary header)	Used to update prevailing ask for Lee-Ready classification
Bid Size	long	Size available at best bid	Optional: DOM Level 1 best bid size
Ask Size	long	Size available at best ask	Optional: DOM Level 1 best ask size
SeqID	long	Exchange-assigned sequence ID for the trade — unique per exchange	exchange_seq_no field. Used for ordering and duplicate detection.
Exchange	string	Exchange code that reported this tick	instrument routing — distinguish same symbol on multiple exchanges
Condition (CID)	string	Correction/Insertion/Deletion flag — exchange-sent trade adjustments	OFE must handle: Correction = update prior tick, Deletion = remove prior tick, Insertion = add between existing ticks
Bid Exchange	string	Exchange code for the best bid quote	Cross-exchange routing for equities
Ask Exchange	string	Exchange code for the best ask quote	Cross-exchange routing for equities

⚠ eSignal timestamps are millisecond precision (not nanosecond). The OFE engine normalises all timestamps to nanoseconds — eSignal timestamps are stored as $\text{ts_ms} \times 1,000,000$ with 000000 nanosecond suffix. This is a known limitation of eSignal vs IQFeed.

⚠ eSignal stocks: the Size field is in ROUND LOTS (1 = 100 shares). The OFE adapter must multiply by 100 to get actual share count. Futures: Size is already in contracts — no multiplication needed.

3.3 eSignal API Connection Options for OFE

API Option	Protocol	Language	Latency	OFE Adapter Use
------------	----------	----------	---------	-----------------

NinjaTrader eSignal Connection	NT built-in adapter	C# (via NT)	~1–5ms	Simplest path: use NT's eSignal connection, same pattern as Kinetick adapter
eSignal Desktop API (COM)	COM/ActiveX	C#, VB, C++	~2–10ms	Direct API access without NT. Windows only. Requires eSignal running.
ICE Consolidated Feed API (C++)	TCP/IP binary	C, C++, Java	< 1ms	Lowest latency. Direct ICE feed. Enterprise-grade. Requires ICE agreement.
DDE Link	DDE protocol	Excel, any DDE client	~10–50ms	Not suitable for OFE — too slow and unreliable for tick-by-tick processing
QLink	Custom TCP	Any socket language	~5–20ms	Legacy option. Not recommended for new integrations.

i Recommended OFE eSignal adapter: use NinjaTrader's built-in eSignal connection (same pattern as Kinetick). This avoids building a separate eSignal connection from scratch and leverages NT's existing symbol mapping and instrument management.

3.4 eSignal Symbol Coverage & Exchange Fees

Market	Exchange	Real-Time Available	eSignal Symbol Format	OFE Symbol Example
US Equities	NYSE, NASDAQ, AMEX, CBOE	Yes — real-time plan	SYMBOL (e.g. AAPL, MSFT)	AAPL, MSFT, TSLA
US Futures	CME, CBOT, NYMEX, COMEX	Yes — CME fees apply	ROOT+MONTH+YEAR (e.g. ESZ5)	ESZ5, NQZ5, CLZ5
US Futures (Continuous)	CME, CBOT	Yes	ROOT+#F (e.g. ES#F)	ES#F, NQ#F, CL#F
US Options	CBOE, various	Yes — OPRA fees apply	Standard option format	AAPL 251219C 00150000
Forex	Composite / FXCM	Yes	CURRENCY1/CURRENCY2	EUR/USD, GBP/JPY
European Futures	Eurex, NYSE Liffe, ICE	Yes — intl fees apply	Exchange-specific format	FDAX Z5 (DAX

				futures)
Indexes	S&P, Dow, Nasdaq, VIX	Yes — included	\$INDEX or ^INDEX	\$SPX, \$INDU, \$VIX

4. IQFeed (DTN) — Technical Specification

4.1 What IQFeed Is

IQFeed, operated by DTN (Data Transmission Network), is a Tier One Data Vendor — one of the very few remaining providers that receives the complete, unfiltered exchange feed directly from exchanges. Most competitors bundle or consolidate ticks; IQFeed delivers every single trade and quote as they occur.

- Tier One Data Vendor: receives full exchange feed direct — no filtering, no aggregation, no tick bundling
- True tick-by-tick data — 'You see every quote and every trade from the exchange' (Optimus Futures)
- Quad Redundant ticker plant — built for high-volume modern markets
- Protocol: TCP/IP socket connections — plain text CSV-delimited messages; versioned protocol (current: 6.2)
- Multiple simultaneous ports: Level 1 (port 5009), Level 2 / Depth (port 9200), History/Lookup (port 9100), Admin (port 9300)
- Language agnostic: any language that can open a TCP socket works (C++, Python, Java, C#, Rust...)
- Market By Order (MBO) feed: CME and ICE full order book available from v6.2 (2024)
- Historical tick data: 8 days during RTH; 180 days outside RTH (same as Kinetick)
- Trade Aggressor field: CME and ICE exchanges report aggressor directly; others inferred via Lee-Ready
- TickID: unique identifier per tick per exchange (32-bit unsigned int, not globally unique — exchange-scoped)
- Asset coverage: US/Canadian equities, futures, options, forex, fixed income, indices, ETFs

4.2 IQFeed TCP Port Architecture

Port	Number	Purpose	OFE Use
Level 1 Feed	5009	Real-time streaming Level 1 data: Last, Bid, Ask, Volume, daily stats per symbol. Text-based streaming.	Primary real-time tick feed. Subscribe symbols here for live data.
Level 2 / Market Depth	9200	Real-time Level 2 Depth of Market: full order book by market maker or price level.	Optional: DOM data for Market Depth indicator. Requires separate exchange subscription.
History / Lookup	9100	Historical tick data (HTT), interval data (HIT), daily data (HDT), market summary reports.	Historical backfill on Symbol Worker startup. Replay for backtesting.
Admin	9300	Login/logout, statistics, client stats, protocol version negotiation.	Connection management. Set protocol version.

			Monitor feed health.
Lookup (alternate)	9100	Market Summary Reports (EDS, FDS, 5MS): bulk snapshots of all symbols in a group.	Watchlist bulk status on startup — get snapshot of all symbols at once.

4.3 IQFeed Level 1 Tick Fields — Complete List

IQFeed Level 1 delivers the richest set of real-time fields of any of the three providers. The following are the fields available via the Level 1 feed (port 5009), sourced directly from the IQFeed API documentation:

4.3.1 Trade / Last Fields

IQFeed Field	Description	OFE Mapping	Critical for OFE ?
Last	Last eligible trade price (excludes extended hours and corrected trades)	price in TickRecord — primary price	YES
Last Size	Volume of the most recent trade	volume in TickRecord	YES
Last Time	Timestamp of the most recent trade (HH:MM:SS format)	exchange_ts — combine with Last Date for full datetime	YES
Last Date	Date of the most recent trade (MM/DD/YYYY)	Combined with Last Time for exchange_ts	YES
Most Recent Trade	Most recent trade price including all trade types (extended hours, corrections)	Alternative to Last — includes more trade types	OPTIONAL
Most Recent Trade Size	Volume of most recent trade (all types)	Extended hours volume tracking	OPTIONAL
Most Recent Trade Time	Timestamp of most recent trade (all types)	Extended hours timestamp	OPTIONAL
Most Recent Trade Aggressor	0=Unknown, 1=Buyer aggressor, 2=Seller aggressor (CME & ICE only)	DIRECT bid/ask classification — eliminates need for Lee-Ready on CME/ICE	CRITICAL for futures
Most Recent Trade Conditions	Trade condition codes (e.g. Form T for extended hours, X for cross)	Filter out non-standard trades from OFE calculation	IMPORTANT

Most Recent Trade Market Center	Exchange/market center that reported the trade	Multi-exchange routing for equities	OPTIONAL
Most Recent Trade Day Code	'D' for regular session, 'X' for non-standard, etc.	Session filter — exclude non-regular-session trades if RTH_ONLY	IMPORTANT
TickID	Exchange-specific tick sequence number (32-bit uint)	exchange_seq_no — ordering and duplicate detection	YES
Total Volume	Total session volume reported by exchange (includes implied trades)	Session volume counter — NOT derived from tick sum	YES
Extended Trade	Extended hours (pre/post market) trade price	Pre/post-market price tracking	OPTIONAL

4.3.2 Quote Fields (Bid / Ask)

IQFeed Field	Description	OFE Mapping
Bid	Current best bid price	Prevailing bid — used in Lee-Ready classification
Ask	Current best ask price (Offer)	Prevailing ask — used in Lee-Ready classification
Bid Size	Number of contracts/shares at the best bid	DOM Level 1 bid size
Ask Size	Number of contracts/shares at the best ask	DOM Level 1 ask size
Bid Time	Timestamp of the most recent bid quote update	Bid quote freshness tracking
Ask Time	Timestamp of the most recent ask quote update	Ask quote freshness tracking
Bid Change	Change in bid price from prior bid	Quote movement tracking
Ask Change	Change in ask price from prior ask	Quote movement tracking
Bid Market Center	Exchange that reported the best bid	Multi-exchange NBBO tracking for equities
Ask Market Center	Exchange that reported the best ask	Multi-exchange NBBO tracking for equities
Spread	Ask - Bid	Liquidity monitoring — wide spread = thin market

4.3.3 Summary / Reference Fields

IQFeed Field	Description	OFE Usage
VWAP	Session VWAP reported by IQFeed/exchange	Cross-check against OFE's own VWAP calculation
High / Low	Session high and low prices	Session extreme tracking
Open / Close	Session open and previous close prices	Opening price for bar context; previous close for gap analysis
Open Interest	Outstanding contracts (futures/options)	Context for futures — large OI = significant level
Settle / Settlement Date	Settlement price and date (futures)	Rollover management
Days to Expiration	Days until contract expiry (futures/options)	Rollover warning for futures
Market Capitalization	Market cap of the equity	Context for stock OFE — large cap behaves differently
Exchange ID	Numeric exchange identifier	Symbol routing and exchange fee tracking
Delay	Minutes of data delay (0 = real-time)	Alert if > 0 — means user does not have real-time subscription for this symbol

4.4 IQFeed Protocol — Message Format

IQFeed communicates via plain text TCP messages. Understanding the protocol is essential for the IQFeed Adapter:

4.4.1 Subscribing to a Symbol (Level 1)

```
w,AAPL\r\n      // Watch symbol AAPL – start streaming
w,MSFT\r\n      // Watch MSFT simultaneously
w,ES#F\r\n      // Watch ES continuous futures contract
r,AAPL\r\n      // Remove AAPL from watch list
```

4.4.2 Incoming Level 1 Update Message (Q message)

```
Q,AAPL,182.47,,,,,182.46,182.48,1,1,,,,,18247350,,,,,,+0.73,,,,,
```

Q messages are comma-delimited. Field order is determined by the S,SELECT UPDATE FIELDS command. Default field order after S,SET PROTOCOL,6.2:

```
Q,Symbol,Exchange ID,Last,Change,
%Change,TotalVol,High,Low,Bid,Ask,BidSz,AskSz,Tick,...
```

4.4.3 Incoming Trade Message (T message) — Trade-Only Mode

```
T,AAPL,2026-05-30 14:32:07,182.4700,200,18247350,1,D,0,,,
```

Field order: Symbol, DateTime, Last, LastSize, TotalVolume, TickID, DayCode, TradeAggressor, TradeConditions, MarketCenter

4.4.4 Protocol Setup Commands

```
S,SET PROTOCOL,6.2\r\n      // Use latest protocol – MUST send first
S,SELECT UPDATE FIELDS,Symbol,Last,Last Size,Last Time,Last Date,Bid,Ask,Bid
Size,Ask Size,Most Recent Trade Aggressor,TickID,Total Volume,Most Recent Trade
Conditions,Most Recent Trade Day Code\r\n
S,TIMESTAMPED MESSAGES,1\r\n  // Include timestamps in all messages
```

i The `S,SELECT UPDATE FIELDS` command is critical. It reduces bandwidth by only requesting the fields the OFE engine needs. For 500 symbols, unnecessary fields waste significant bandwidth and parsing time.

4.4.5 Historical Tick Request (port 9100)

```
HTT,AAPL,20260530 09:30:00,20260530 16:00:00,,,,1\r\n
```

HTT = Historical Tick by Tick. Parameters: Symbol, BeginDateTime, EndDateTime, MaxDatapoints, BeginFilterTime, EndFilterTime, DataDirection (1=newest first, 0=oldest first)

```
Response per tick: LH,AAPL,2026-05-30
09:30:01.247,182.35,100,18100000,12847,Normal,,
```

LH message: DataMessageID, Symbol, DateTime(ms), Last, LastSize, TotalVolume, TickID, Conditions, MarketCenter

4.5 IQFeed Trade Aggressor — Key Advantage

IQFeed's Trade Aggressor field is a major advantage for order flow analysis on futures markets:

Scenario	IQFeed Aggressor Field	OFE Classification	Accuracy
CME/ICE futures trade — exchange reports aggressor	Most Recent Trade Aggressor = 1 (Buyer) or 2 (Seller)	Direct: 1=ask_vol, 2=bid_vol. No Lee-Ready needed.	100% accurate — exchange-confirmed
NYSE/NASDAQ equity — exchange does not report aggressor	Most Recent Trade Aggressor = 0 (Unknown)	Use Lee-Ready: compare Last to prevailing Bid/Ask	~97-99% accurate — standard industry method
Tick between bid and ask (midpoint trade)	Aggressor = 0, price = (bid+ask)/2	Lee-Ready: tick direction (uptick=ASK, downtick=BID)	~90% accurate — ambiguous by nature

i CME futures (ES, NQ, CL, GC, ZB) are the most important for order flow analysis. IQFeed provides direct aggressor classification for all of them — making Lee-Ready unnecessary for the most commonly traded instruments. This is a significant quality advantage over Kinetick and eSignal.

4.6 IQFeed Best Practices for OFE (from official documentation)

The IQFeed API documentation explicitly recommends a three-threaded approach — which aligns perfectly with the OFE Symbol Worker design:

- Thread 1 — Receive: dedicated thread reading from TCP socket into a ring buffer (OFE: Tick Router)
- Thread 2 — Process: parses raw messages into TickRecord structs (OFE: Symbol Worker)
- Thread 3 — Act: executes calculations, signal detection (OFE: remaining Symbol Worker pipeline)
- Use S,SELECT UPDATE FIELDS to request only needed fields — critical for bandwidth at 500+ symbols
- Use multiple connections: up to 50 historical requests/second on port 9100
- Optimise receive buffer size per language — larger buffer reduces system call overhead
- Request bulk historical data outside RTH (9:30–4:00 ET) to avoid the 8-day tick limit
- Monitor queueing via Admin port — if KB Queued > 0, processing is falling behind the feed

5. Universal Tick Record — Provider-Agnostic Internal Format

All three providers deliver tick data in different formats, field names, and protocols. The OFE engine operates on a single internal Universal Tick Record. Each provider's adapter normalises its native format into this record before the tick reaches the Symbol Worker.

Field	Type	Description	Kinetick Source	eSignal Source	IQFeed Source
symbol_id	uint32	Internal symbol hash — fast routing key	Instrument.Full Name hash	Symbol string hash	Symbol string hash
price	double	Executed price (full precision)	Price field	Last / Price field	Last field
volume	int64	Contracts/shares at this tick	Volume field	Size × multiplier	Last Size field
side	enum BID/ASK/UNKNOWN	Aggressor side classification	Lee-Ready on Last tick	Lee-Ready on Trade tick	Aggressor field (CME/ICE direct) or Lee-Ready
exchange_ts_ns	int64	Exchange timestamp in Unix nanoseconds	Time → ns	Time (ms) × 1,000,000	Last DateTime → ns
receive_ts_ns	int64	OFE engine receipt time in Unix nanoseconds	System clock on receipt	System clock on receipt	System clock on receipt
exchange_seq_no	uint64	Exchange sequence number	TickCount (internal only)	SeqID field	TickID field
tick_type	enum TRADE/BID_QUOTE/ASK_QUOTE/SUMMARY	Type of this market event	MarketDataType enum	Type field from T&S	Q message type field
trade_conditions	string	Exchange trade condition codes	Not provided by NT/Kinetick	Condition (CID) field	Most Recent Trade Conditions
day_code	char	Session indicator: D=regular, X=non-standard	Not directly provided	Not directly provided	Most Recent Trade Day Code

provider_id	enum KINETICK/E SIGNAL/IQF EED	Which data provider sent this tick	Hardcoded KINETICK	Hardcoded ESIGNAL	Hardcode d IQFEED
raw_bytes	bytes (optional)	Original raw message for audit/debugging	Serialised NT event	Serialised COM event	Original CSV line

✓ All Symbol Worker code operates only on Universal Tick Records. Changing from Kinetick to IQFeed requires only replacing the adapter — zero changes to the Symbol Worker, Signal Detector, or any other engine component.

6. Provider Adapter Architecture

6.1 Adapter Interface (C++ Abstract Base)

```
class IDataFeedAdapter {
public:
    // Called once on startup – open connection, authenticate, subscribe initial
    symbols
    virtual bool Connect(const AdapterConfig& config) = 0;

    // Subscribe one symbol – adapter starts streaming ticks for this symbol
    virtual bool Subscribe(const std::string& symbol) = 0;

    // Unsubscribe symbol – adapter stops streaming
    virtual bool Unsubscribe(const std::string& symbol) = 0;

    // Request historical tick backfill for symbol from from_ts to to_ts
    virtual void RequestHistory(const std::string& symbol,
                               int64_t from_ts_ns, int64_t to_ts_ns,
                               HistoryCallback callback) = 0;

    // Output queue – Symbol Workers read from this
    virtual MPMCQueue<UniversalTickRecord>* GetOutputQueue() = 0;

    // Health check – returns latency and connection status
    virtual AdapterStatus GetStatus() = 0;

    virtual void Disconnect() = 0;
};
```

Three concrete implementations: KinetickAdapter (C# via NT bridge), ESignalAdapter (COM or NT bridge), IQFeedAdapter (direct TCP socket in C++).

6.2 Provider Comparison — OFE Adapter Complexity

Aspect	Kinetick Adapter	eSignal Adapter	IQFeed Adapter
Implementation language	C# (inside NinjaTrader AddOn)	C# (via NT) or C++ (ICE API)	C++ direct TCP — any language
Connection method	NT built-in — no separate socket	NT built-in or COM API call	TCP socket to localhost IQConnect.exe
Symbol subscription	AddDataSeries() per instrument	NT Subscribe() or COM Watch()	w,SYMBOL\r\n per socket
Protocol parsing	NT handles — OFE reads NT events	NT handles OR parse COM events	Parse CSV messages from TCP stream
Timestamp precision	Milliseconds (NT	Milliseconds	Milliseconds (Last Time

	DateTime)		field) — no nanoseconds
Aggressor classification	Lee-Ready required for all	Lee-Ready required for all	CME/ICE: direct. Others: Lee-Ready.
Historical data access	Via NT Historical Data Manager	Via NT or eSignal API (40-day ext.)	Direct port 9100 TCP requests
Multi-symbol efficiency	NT manages subscriptions	NT manages OR separate COM per symbol	Single TCP connection handles ALL symbols
Adapter development effort	Medium — NT AddOn framework	Medium — NT or COM	Low — simple TCP socket + CSV parser
Runtime dependency	NinjaTrader 8 must be running	NinjaTrader 8 OR eSignal running	IQConnect.exe must be running (Windows)

7. Configurable Symbol Count — 1 to 1000+

The user's correction is exactly right: the engine must accommodate any number of symbols from 1 to 1000+ without architectural changes. The symbol count is a runtime configuration parameter, not a compile-time constant. The engine scales automatically.

7.1 Symbol Count Configuration

Symbol count is configured in the engine's YAML configuration file and can be changed at runtime:

```
# ofe_engine.yaml – master configuration
engine:
  max_symbols: 500          # Maximum symbols the engine will accept
  # Valid range: 1 to 10000 (hardware-dependent upper limit)
  # 1      = single symbol mode (minimal resources)
  # 10     = small watchlist
  # 100    = standard multi-symbol
  # 500    = large watchlist
  # 1000+  = enterprise / scanning mode

data_feed:
  provider: IQFEED          # KINETICK | ESIGNAL | IQFEED
  primary_provider: IQFEED
  failover_provider: KINETICK # Automatic failover if primary drops
  kinetick_login: 'user@email.com'
  kinetick_password: 'xxxx'
  iqfeed_login: 'XXXXXX'
  iqfeed_password: 'xxxx'
  iqfeed_product_id: 'OFE_ENGINE_1'
  iqfeed_version: '1.0.0'
  esignal_com_server: 'localhost'

resources:
  worker_threads: auto      # auto = one thread per CPU core
  # or set explicitly: worker_threads: 16
  ring_buffer_size: 10000  # ticks per symbol ring buffer
  memory_per_worker_mb: 2  # MB allocated per Symbol Worker
```

7.2 How Symbol Count Affects Resource Allocation

Symbol Count	Worker Threads Used	RAM Required	Tick Buffer Total	Recommended CPU
1	1 (single worker)	~2 MB	10,000 ticks	Any modern CPU
10	1–4 (shared cores)	~20 MB	100,000 ticks	2-core minimum
50	4–8 (shared cores)	~100 MB	500,000 ticks	4-core recommended

100	8–16	~200 MB	1,000,000 ticks	8-core recommended
250	16–32	~500 MB	2,500,000 ticks	16-core recommended
500	32–64	~1 GB	5,000,000 ticks	32-core recommended
1000	64–128	~2 GB	10,000,000 ticks	64-core / 2 servers
5000	Distributed across multiple servers	~10 GB	50,000,000 ticks	Kubernetes cluster

i 'auto' thread mode: the engine detects available CPU cores at startup and assigns one Symbol Worker thread per core, multiplexing multiple symbols per thread when symbol count exceeds core count. With 8 cores and 100 symbols: each core handles ~12 symbols time-shared via a work-stealing scheduler.

7.3 Dynamic Symbol Addition & Removal

Symbols can be added or removed from the active watchlist at runtime without restarting the engine:

Operation	API Call	Engine Behavior	Time to Activate
Add symbol	POST /v1/watchlist/{id}/symbols { symbol: 'NVDA' }	1. Allocate Symbol Worker (or assign to existing thread). 2. Initialise state. 3. Request historical backfill from provider. 4. Subscribe live feed. 5. Start streaming.	3–10 seconds (backfill time dominant)
Remove symbol	DELETE /v1/watchlist/{id}/symbols/NVDA	1. Unsubscribe live feed from provider. 2. Flush pending ticks. 3. Save state to DB. 4. Deallocate Symbol Worker. 5. Release memory.	< 1 second
Replace watchlist	PUT /v1/watchlist/{id}/symbols [...]	Compute diff: symbols to add + remove. Execute both operations concurrently.	Max activation time of any new symbol
Clear all symbols	DELETE /v1/watchlist/{id}/symbols	Unsubscribe all. Keep watchlist entity (empty). Engine idle.	< 5 seconds
Pause symbol	PATCH /v1/watchlist/{id}/symbols/NVDA { active: false }	Keep subscription active (ticks received) but suspend signal detection. Saves CPU. Ticks still buffered.	Instant

7.4 Multi-Provider Configuration — Same Symbol from Different Providers

For maximum resilience and data quality, the engine supports using multiple providers simultaneously:

```
watchlist:
- watchlist_id: futures_watchlist
  provider: IQFEED          # IQFeed for futures – best aggressor data
  symbols: [ES#F, NQ#F, CL#F, GC#F, ZB#F]

- watchlist_id: stocks_watchlist
  provider: KINETICK        # Kinetick for stocks via NinjaTrader
  symbols: [AAPL, MSFT, NVDA, TSLA, AMZN, META, GOOGL]

- watchlist_id: esignal_global
  provider: ESIGNAL         # eSignal for international markets
  symbols: [FDAX Z5, FGBL Z5, SX5E] # European futures
```

***i** Each watchlist can use a different provider. The Tick Router combines all streams into a single unified flow to Symbol Workers. Symbol Workers don't know or care which provider delivered their ticks — they only see Universal Tick Records.*

7.5 Provider Failover Configuration

If the primary provider disconnects, the engine automatically switches to the failover provider with zero manual intervention:

```
data_feed:
  primary: IQFEED
  failover: KINETICK
  failover_trigger_ms: 5000    # Switch if no ticks for 5 seconds
  failover_log: true          # Log all failover events
  auto_reconnect_primary: true # Reconnect primary when it recovers
  reconnect_delay_s: 30       # Wait 30s before reconnecting primary
```

Failover Scenario	Engine Behavior
IQFeed disconnects (network issue)	After 5s silence: start Kinetick subscriptions for all active symbols. Log gap in tick history. Continue processing.
eSignal API crashes	Adapter detects COM exception. Switch to backup provider immediately. Alert developer via webhook.
Kinetick NT connection drops	NT fires OnConnectionStatusChange(). Adapter signals OFE. Switch to IQFeed if configured.
Primary reconnects	After failover, engine monitors primary. When healthy, gracefully switches back. Gap is noted in tick history with provider_switch_ts.

Both providers down	Engine continues with last known bar state. No new ticks. Signal detection pauses. Alerts fire. Historical data still accessible.
---------------------	---

8. Complete Engine Configuration Reference

All engine parameters in the master YAML configuration file. Every parameter has a sensible default. A minimal single-symbol, single-provider deployment requires only 3 lines of config.

Config Key	Type	Default	Valid Range	Description
engine.max_symbols	int	100	1 – 10000	Maximum symbols the engine will simultaneously monitor
engine.worker_threads	string/int	auto	auto or 1–256	CPU threads for Symbol Workers. auto = one per core.
engine.ring_buffer_size	int	10000	1000 – 1000000	Ticks per symbol ring buffer. Larger = more memory, fewer drops under burst.
engine.session_filter	enum	RTH_ONLY	ALL / RTH_ONLY / ETH_ONLY / GLOBEX	Default session filter for all symbols
engine.log_level	enum	INFO	DEBUG / INFO / WARN / ERROR	Engine log verbosity
data_feed.provider	enum	KINETICK	KINETICK / ESIGNAL / IQFEED / MULTI	Primary tick data provider
data_feed.failover_provider	enum	null	Same as provider or null	Backup provider on primary disconnect
data_feed.failover_trigger_ms	int	5000	1000 – 60000	Milliseconds of silence before triggering failover
data_feed.iqfeed_host	string	localhost	Any hostname/IP	IQConnect.exe host (usually localhost)
data_feed.iqfeed_level1_port	int	5009	1024–65535	IQFeed Level 1 streaming port
data_feed.iqfeed_history_port	int	9100	1024–65535	IQFeed History/Lookup port
data_feed.iqfeed_protocol	string	6.2	4.9, 5.1, 5.2, 6.1,	IQFeed protocol

			6.2	version
data_feed.esignal_host	string	localhost	Any hostname	eSignal COM server host
data_feed.kinetick_via_nt_port	int	7777	1024–65535	Local TCP port for Kinetick NT bridge
indicators.imbalance_ratio	float	3.0	1.5 – 10.0	Ratio for imbalance detection
indicators.stacked_min	int	3	2 – 10	Minimum consecutive imbalances for stacked zone
indicators.value_area_pct	float	0.70	0.60 – 0.90	Value area percentage for Volume Profile
indicators.absorption_vol_threshold	int	500	50 – 100000	Minimum volume for absorption detection
indicators.enable_pulse	bool	true	true/false	Enable Pulse signal detection
indicators.enable_turns	bool	true	true/false	Enable Turns signal detection
indicators.enable_vwap	bool	true	true/false	Enable VWAP calculations
api.rest_port	int	8080	1024–65535	REST API listen port
api.websocket_port	int	8081	1024–65535	WebSocket listen port
api.auth_enabled	bool	true	true/false	Require API key authentication
api.rate_limit_per_min	int	1000	0 (unlimited) – 100000	REST API rate limit per key
storage.timescaledb_url	string	required	Valid JDBC URL	TimescaleDB connection string for tick/signal storage
storage.redis_url	string	required	Valid Redis URL	Redis for live state cache and event bus

9. Provider Selection Guide — Which to Use When

Use Case	Recommended Provider	Reason
NinjaTrader 8 futures (ES, NQ, CL) day trading	Kinetick	Tightest NT integration; Valtos's personal recommendation; CME exchange fees reduced; zero adapter complexity
Futures order flow with best aggressor accuracy	IQFeed	Direct trade aggressor field for CME and ICE — eliminates Lee-Ready approximation; true tick-by-tick; 64-bit TickID for ordering
Multi-symbol stock scanning (100+ US equities)	IQFeed	Single TCP connection handles all symbols efficiently; broad NYSE/NASDAQ coverage; true tick-by-tick; market summary reports for bulk status
International markets (European futures, LSE, TSX)	eSignal	95%+ global exchange coverage via ICE Consolidated Feed; best international data quality
Maximum resilience / production system	IQFeed primary + Kinetick failover	IQFeed for quality; Kinetick as NT-native backup if IQFeed drops
Lowest cost / simple setup for learning	Kinetick (EOD free tier)	Free historical data built into NT; simple; sufficient for strategy development and backtesting
Algorithmic development requiring API access	IQFeed	Language-agnostic TCP API; extensive documentation; large developer community; used by many algo systems

10. Summary — Key Engineering Decisions

Based on complete research of all three providers, the following engineering decisions are recommended:

Decision	Recommendation	Rationale
Primary adapter for NinjaTrader v1.0	Kinetick (via NT bridge)	Simplest integration; already in NT8; Valtos's preference; no separate IQConnect needed for NT users
Primary adapter for standalone / API users	IQFeed (direct TCP)	Best tick quality; direct aggressor; language-agnostic; most developer-friendly API
eSignal adapter	Phase 2 (post-launch)	Lower priority: more complex; higher cost; primarily needed for international markets
Universal Tick Record	REQUIRED — implement first	All adapters normalise to same struct. Adding providers never changes Symbol Worker code.
Trade Aggressor classification	IQFeed: use direct field for CME/ICE. Kinetick/eSignal: use Lee-Ready.	Maximises accuracy where possible; falls back to standard method where direct not available
Symbol count	Runtime config parameter: 1 to 10000	No hardcoded limits anywhere in engine. Resource allocation scales automatically by symbol count.
Multi-provider simultaneous	Support from v1.0	Different watchlists can use different providers. Futures on IQFeed, stocks on Kinetick, international on eSignal.
Timestamp normalisation	All to Unix nanoseconds (int64)	All three providers give $\leq$ millisecond precision. Nanosecond storage is forward-compatible.
Historical backfill on symbol activation	Always backfill from provider history port	Symbol Worker must have today's intraday bars before serving live signals. Critical for Delta and Volume Profile accuracy.

END OF DOCUMENT — Data Feed Integration & Configurable Multi-Symbol Architecture Spec v1.0

Covers: Kinetick · eSignal · IQFeed technical specs · Universal Tick Record · Adapter design · 1-to-1000 configurable symbol count · Provider failover · Complete config reference